回顾：数据结构基本概念

■ 数据结构：相互之间存在一种或多种特定关系的数据元素的集合

■ **数据**：对信息的一种**符号**表示，指所有能输入到计算机中进行程序处理的符号的总称

✓ 可以输入计算机，能被计算机处理，包括整型、实型等数值类型，声音、图像、视频等作为符号类型……

■ **数据元素**：数据的基本单位，通常作为整体处理，也被称为记录。一个数据元素可由若干数据项组成

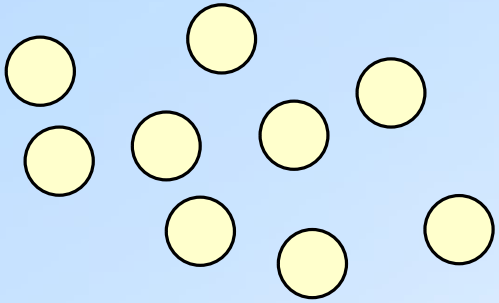
■ **数据项**：组成数据元素的具有独立含义的最小单位

数据元素		
数据项1	数据项2	数据项3

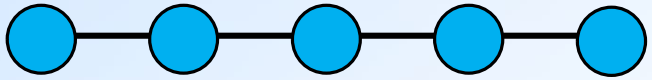
人					
眼睛	手	脚	嘴巴	耳	…

回顾：数据结构基本概念

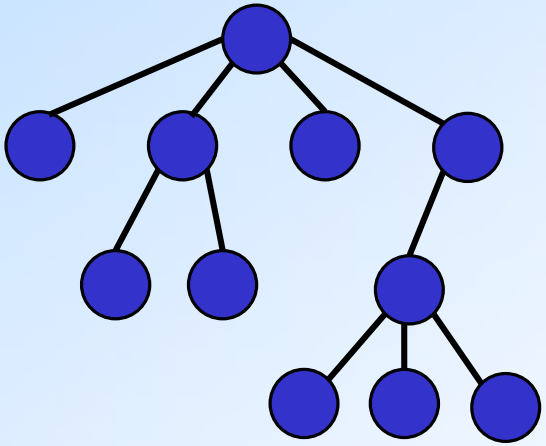
- 四种基本结构



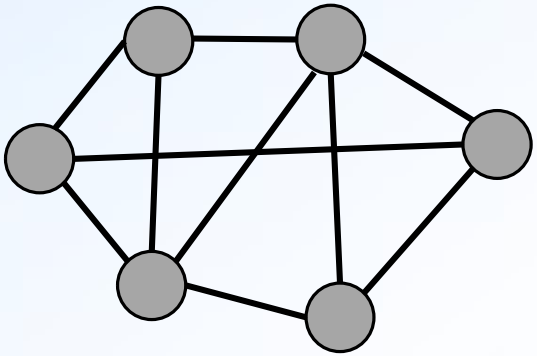
集合结构



线性结构



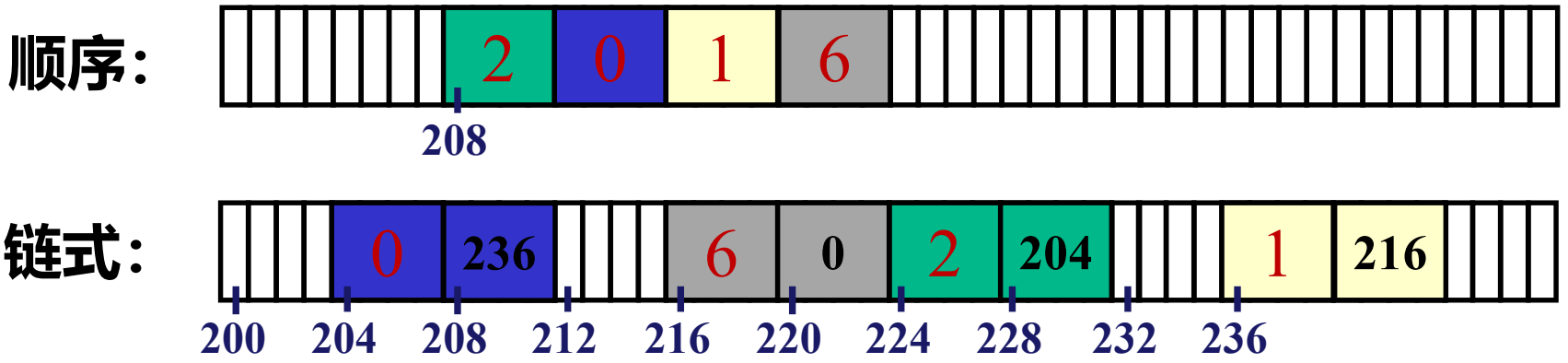
树形结构



图形结构

回顾：数据结构基本概念

- 数据的存储结构
  - ✓ 数据结构在计算机存储器中的存储映像，又叫物理结构
  - ✓ 存储结构既要反应数据元素本身，还要反映数据元素之间的关系
  - ✓ 一般有两种存储结构：顺序存储结构与链式存储结构



  - ✓ 索引存储：在存储元素信息的同时建立附加的索引表
  - ✓ 散列存储：根据节点的关键码通过一个散列函数计算得到存储地址

回顾：性能分析与度量

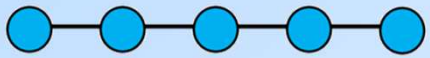
算法的性能标准

- 正确性：包括　不含语法错误
  对几组数据运行正确
  对典型、苛刻的数据运行正确；
  对所有数据运行正确
- 可读性
- 效率：高效、低存储需要。（算法执行时间短，同时所占用的存储空间小。
- 健壮性：当输入非法数据时，算法也能作出适当反应，而不会出现莫名其妙的输出结果。

第二章 线性表

- 线性表
- 顺序表
- 链表
- 顺序表与链表的比较

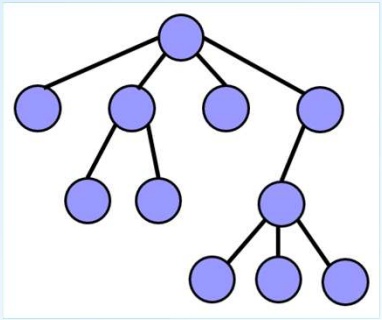
数据结构分类

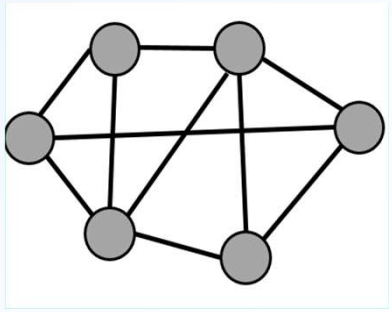


- 数据结构
  - 线性结构
    - 线性表
      - 顺序表
      - 链表

存储结构分类

- 数据结构
  - 非线性结构





逻辑结构分类

线性表

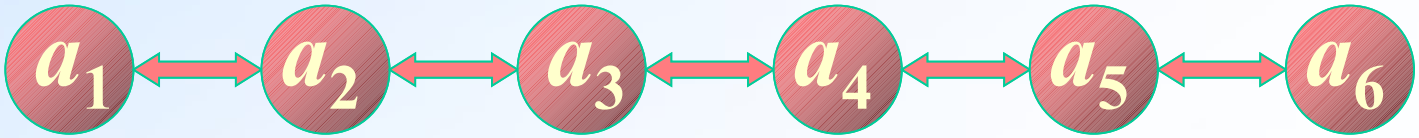
定义： n（≥0）个数据元素的有限序列，记作

$$(a_1, \ldots a_{i-1}, a_i, a_{i+1}, \ldots, a_n)$$

其中，a_i 是表中数据元素，n 是表长度。

特点:

- 同一线性表中元素具有相同特性。
- 相邻数据元素之间存在序偶关系。
- 除第一个元素外，其他每一个元素有一个且仅有一个直接前驱。
- 除最后一个元素外，其他每一个元素有一个且仅有一个直接后继。



顺序表

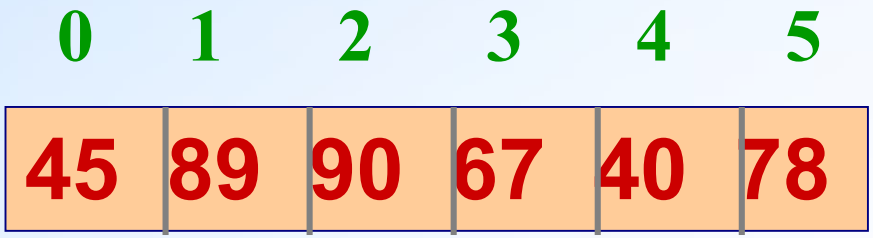
定义： 将线性表中的元素相继存放在一个连续的存储空间中。

存储结构： 数组。

特点： 线性表的顺序存储方式。

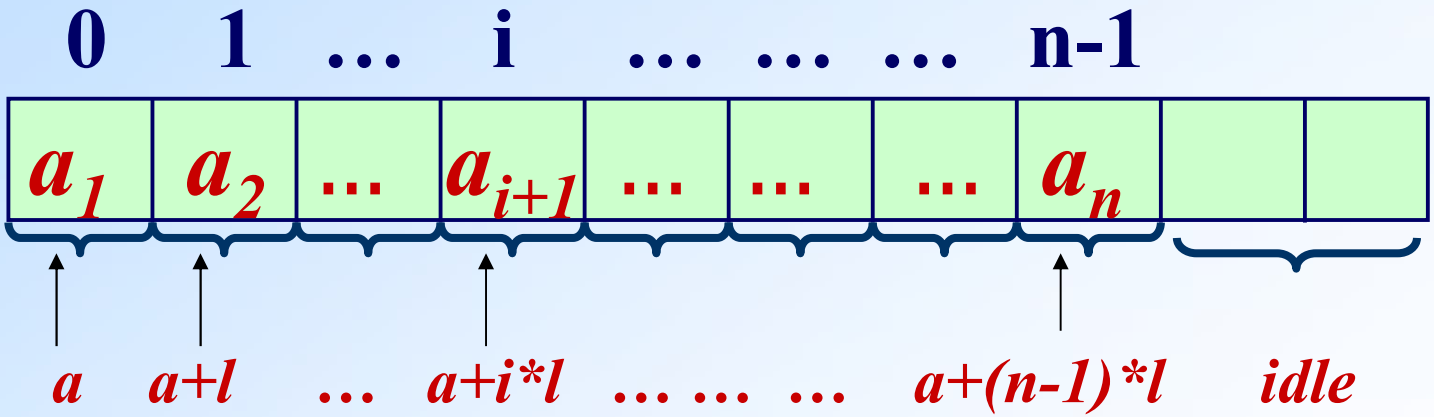
存取方式： 顺序存取、随机存取

顺序存储结构示意图



顺序表的存储方式：

$$LOC(a_{i+1}) = LOC(a_i)+l$$
$$LOC(a_{i+1}) = LOC(a_1)+i*l$$



假设每个元素所占的字节数为l

顺序表（SeqList）的类型定义

```
#define ListSize 100      //最大允许长度
typedef int               //数据元素为整型
ListData;
typedef struct {          顺序表定义为一个结构体
  ListData * data;        //存储空间基址
  int length;             //当前元素个数
} SeqList;
```

顺序表基本运算

引用作为函数参数进行传递时，实质上传递的是实参本身，对形参的修改其实是对实参的修改。

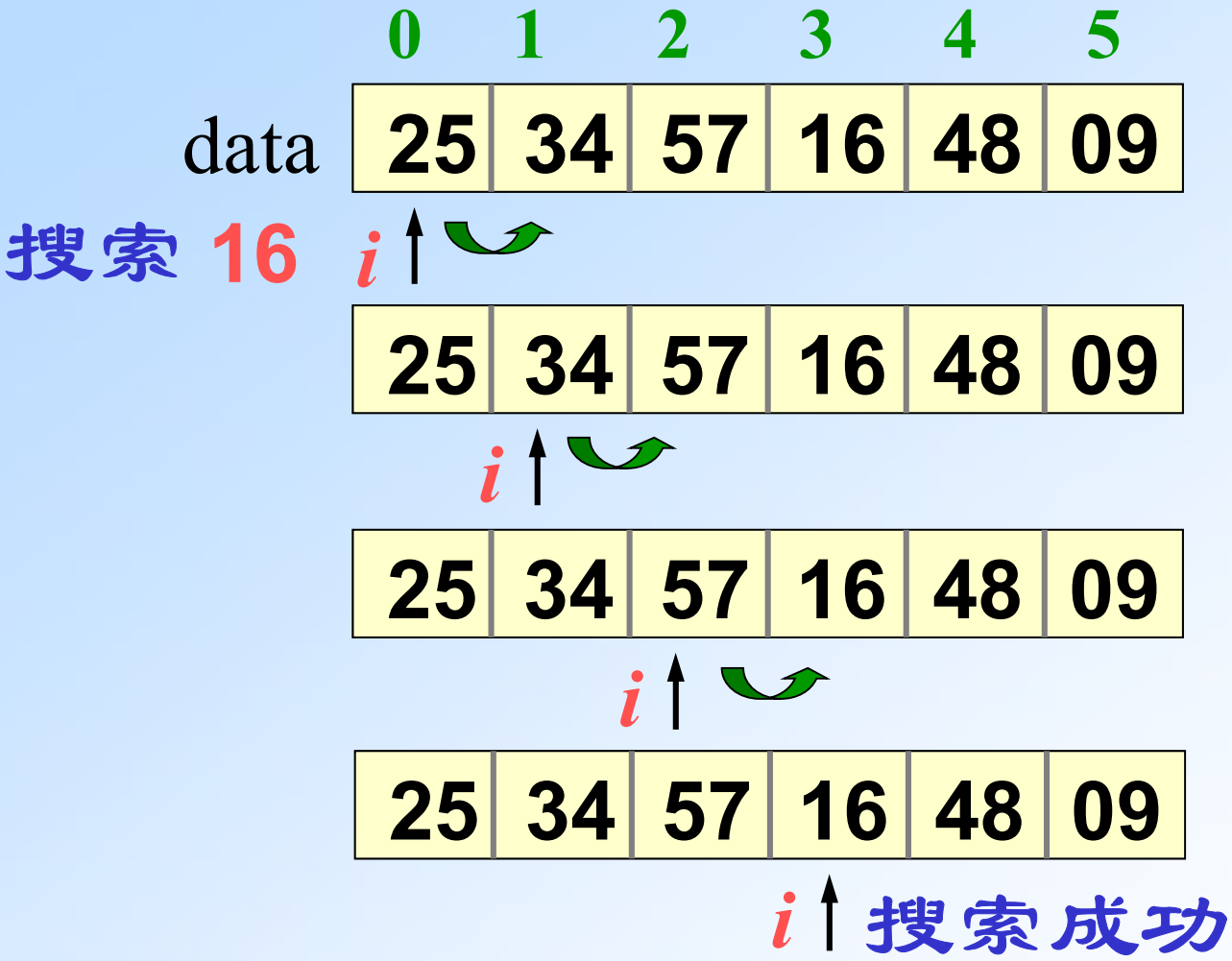
- **初始化**

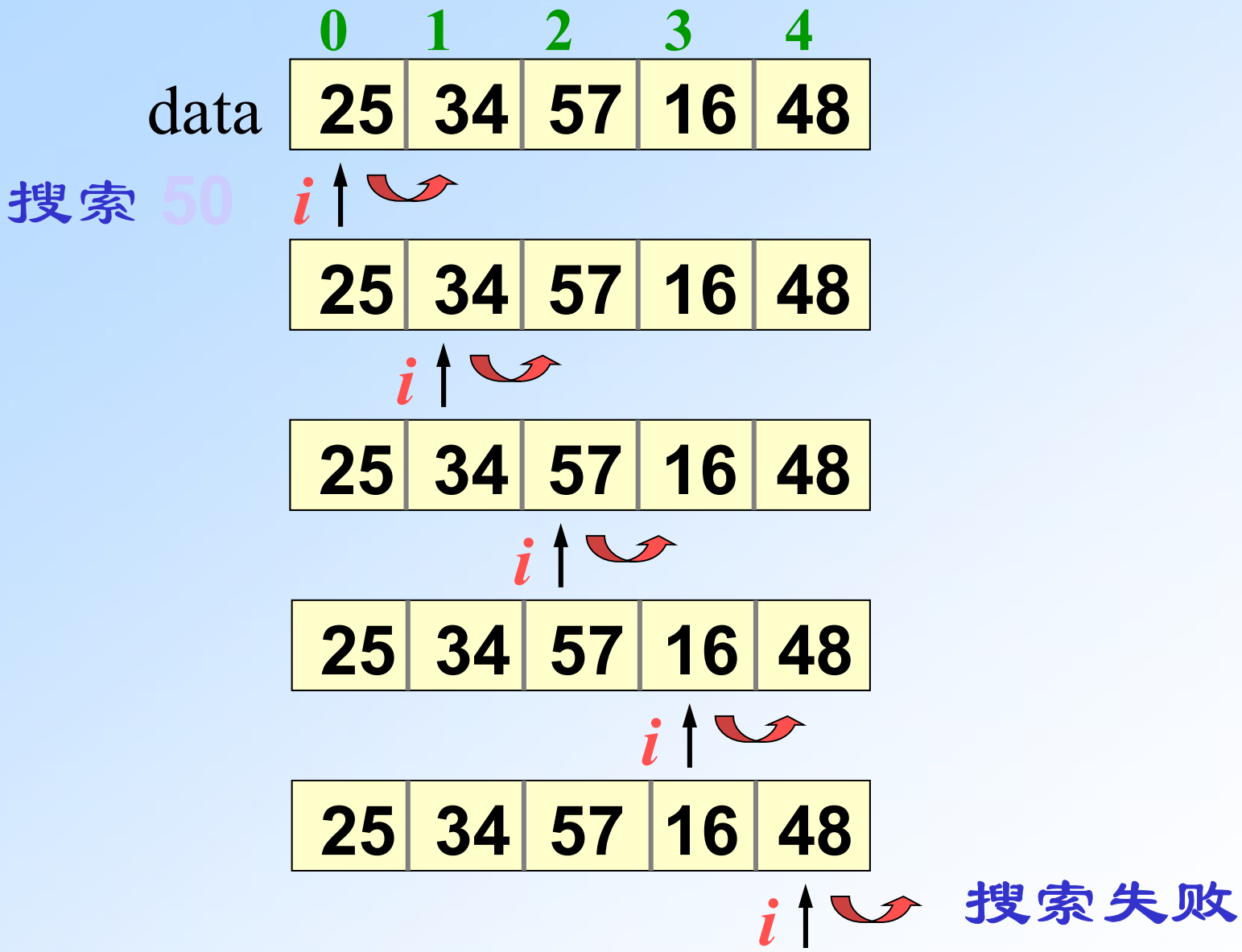
```
void InitList ( SeqList & L ) {
    L.data = ( ListData * ) malloc
        ( ListSize * sizeof ( ListData ) );
    if ( L.data == NULL ) {
        printf ( "存储分配失败!\n" );
        exit (1);
    }
    L.length = 0;
}
```

- **按值查找**：找x在表中的位置，若查找成功，返回表项的位置（数组下标），否则返回-1

```
int Find ( SeqList &L, ListData x ) {
    int i = 0;
    while ( i < L.length && L.data[i] != x )
        i++;
    if ( i < L.length ) return i;
    else return -1;
}
```

顺序搜索图示


0 1 2 3 4 5
data
25 34 57 16 48 09
搜索 16
i
25 34 57 16 48 09
i
25 34 57 16 48 09
i
25 34 57 16 48 09
i 搜索成功


0 1 2 3 4
data
25 34 57 16 48
搜索 50
i
25 34 57 16 48
i
25 34 57 16 48
i
25 34 57 16 48
i
25 34 57 16 48
i
搜索失败

搜索成功的平均比较次数

$$ACN = \sum_{i=0}^{n-1} p_i \times c_i$$

P_i为查找元素i的概率，
c_i为需要比较的次数

若搜索概率相等，则

$$ACN = \frac{1}{n}\sum_{i=0}^{n-1}(i+1) = \frac{1}{n}(1+2+\cdots+n) =$$

$$= \frac{1}{n} * \frac{(1+n)*n}{2} = \frac{1+n}{2}$$

搜索不成功　数据比较 n 次

时间复杂度：最坏O(n)，最好O(1)，平均O(n)

按值查找：判断x是否在表中

```
int IsIn ( SeqList &L, ListData x )
{       int i = 0,    found=0;
        while ( i < L.length &&!found )
                if(L.data[i] != x ) i++;
                else found=1;
        return found;
}
```

- **求表的长度**

```
int Length ( SeqList & L ) {
    return L.length;
}
```

- **提取函数**：在表中提取第 i 个元素的值

```
ListData GetData ( SeqList &L, int i ) {
  if ( i >= 0 && i < L.length )
      return L.data[i];
  else printf ( "参数 i 不合理！ \n" );
}
```

- **按值查找**：寻找x的后继

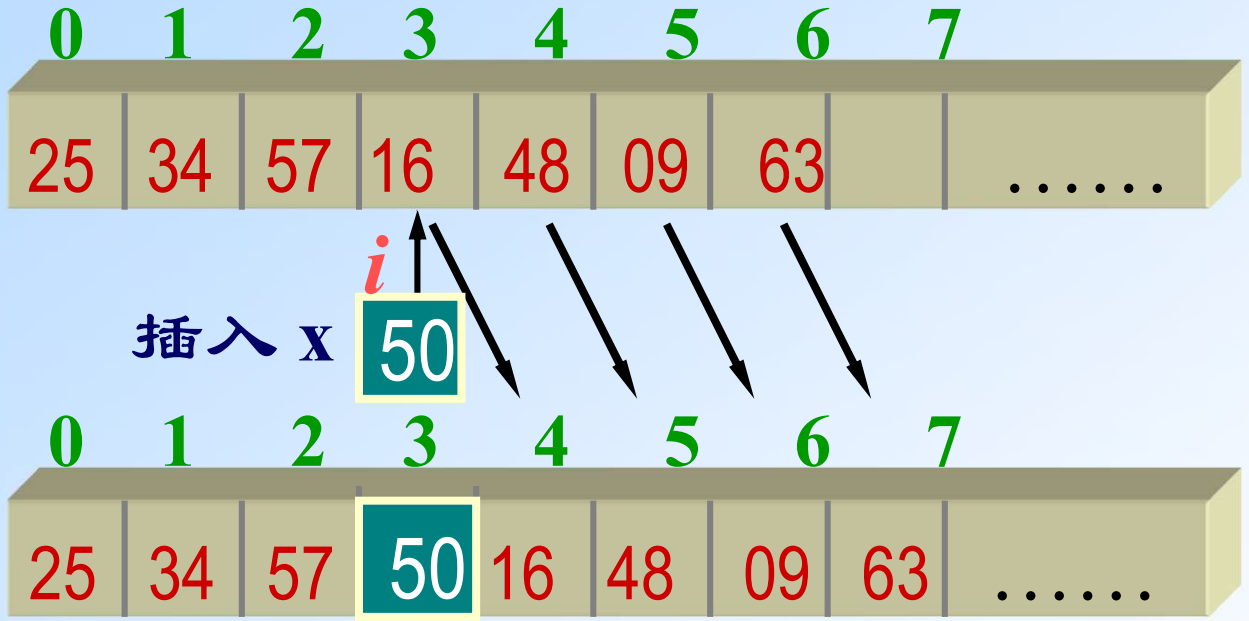
```
int Next ( SeqList &L, ListData x ) {
    int i = Find(x);
    if ( i >=0 && i < L.length-1 ) return i+1;
      else return -1;
}
```

- 寻找x的前驱

```
int Previous ( SeqList &L, ListData x ) {
    int i = Find(x);
    if ( i >0 && i < L.length ) return i-1;
      else return -1;
}
```

- **插入**

共有N+1个位置
可以插入



顺序表插入时，平均数据移动次数AMN在各表项插入概率相等时

$$AMN = \frac{1}{n+1}\sum_{i=0}^{n}(n-i) = \frac{1}{n+1}(n+\cdots+1+0)$$

$$= \frac{1}{(n+1)}\frac{n(n+1)}{2} = \frac{n}{2}$$

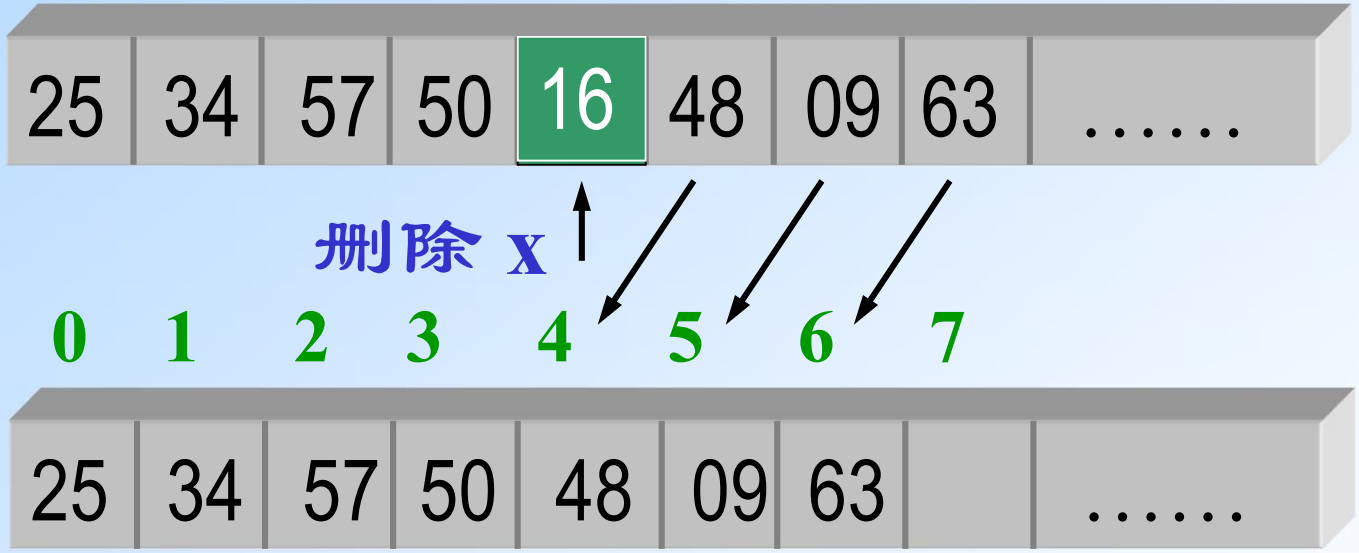
能否自前向后移动?

- **顺序表的插入**

```
int Insert ( SeqList &L, ListData x, int i ) {
//在表中第 i 个位置插入新元素 x
if (i < 0 || i > L.length || L.length == ListSize)
  return 0;              //插入不成功
  else {
        for ( int j = L.length; j > i; j-- )
              L.data[j] = L.data[j -1];
        L.data[i] = x;   L.length++;
        return 1;     //插入成功
    }
  }
```

时间复杂度：最坏O(n)，最好O(1)，平均O(n)

- **删除**



顺序表删除平均数据移动次数AMN在各表项删除概率相等时

$$AMN = \frac{1}{n}\sum_{i=0}^{n-1}(n-i-1) = \frac{1}{n}\frac{(n-1)n}{2} = \frac{n-1}{2}$$

- **顺序表的删除**

```
int Delete ( SeqList &L, ListData x ) {
//在表中删除已有元素 x
int i = Find (L, x);          //在表中查找 x
if ( i >= 0 ) {
      L.length −− ;
      for ( int j = i; j < L.length; j++ )
            L.data[j] = L.data[j+1];
      return 1;                  //成功删除
   }
return 0;                        //表中没有 x
}
```

时间复杂度：最坏O(n)，最好O(1)，平均O(n)

顺序表的应用: 集合的“并”运算

```
void Union ( SeqList &A, SeqList &B ) {
    int n = Length ( A );
    int m = Length ( B );
    for ( int i = 0; i < m; i++ ) {
        int x = GetData ( B, i );  //在B中取一元素
        int k = Find (A, x);       //在A中查找它
        if ( k == -1 )             //若未找到插入它
          { Insert ( A, x, n );  n++; }
    }
}
```

把B中与A不重复的元素插入A中

- **集合的“交”运算**

```
void Intersection ( SeqList &A, SeqList &B ) {
   int n = Length ( A );
   int m = Length ( B );
   int i = 0;
   while ( i < n ) {
      int x = GetData ( A, i );  //在A中取一元素
       int k = Find ( B, x );    //在B中查找它
       if ( k == -1 ) { Delete ( A, i );  n--; }
       else i++;                 //未找到在A中删除它
   }
}
```

把A中与B不重复的去掉

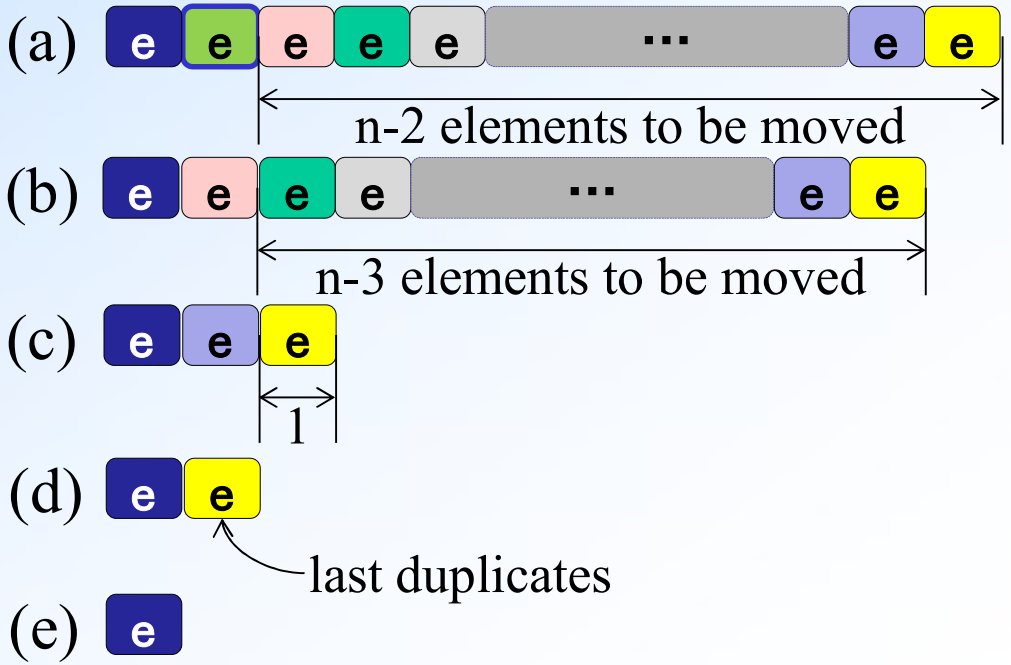
有序向量唯一化（重复元素剔除算法）

■ 唯一化(低效版)

```
template <typename T>
int Vector<T>::uniquify() {                    //有序向量重复元素剔除算法（低效版）
    int oldSize = _size; int i = 1;            //原始向量长度，起始于第二个元素
    while ( i < _size )                        //从前向后，逐一比对各对相邻元素
        _elem[i - 1] == _elem[i] ? remove( i ) : i++;
    return oldSize - _size;                    //向量规模变化量，即被删除元素总数
}
```

每次删除一个后，需要不断地移动

最坏情况：复杂度 $O(n^2)$
未充分利用有序性



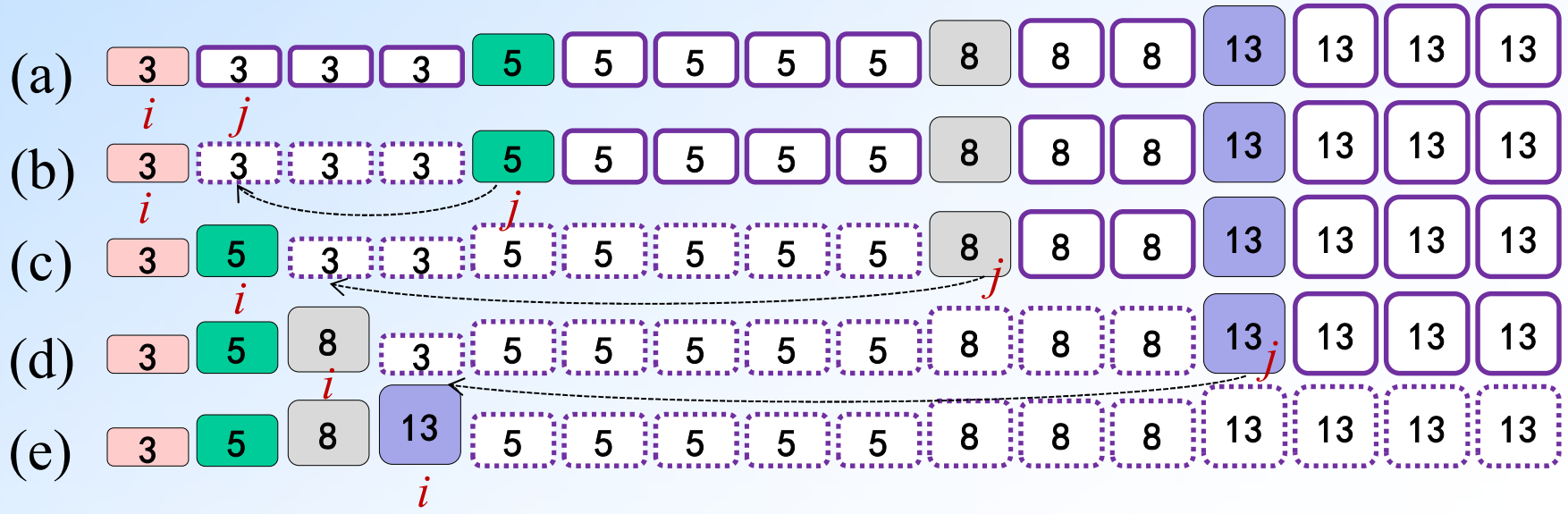
有序向量唯一化（重复元素剔除算法）

■ 唯一化(高效版)

如何实现时间复杂度O(n)?

```
template <typename T>
int Vector<T>::uniquify() {                  //有序向量重复元素剔除算法（高效版）
   Rank i = 0, j = 0;                        //各对互异“相邻”元素的秩
   while ( ++j < _size )                     //逐一扫描，直至末元素
      if ( _elem[i] != _elem[j] )            //跳过雷同者
         _elem[++i] = _elem[j];              //发现不同元素时，向前移至紧邻于前者右侧

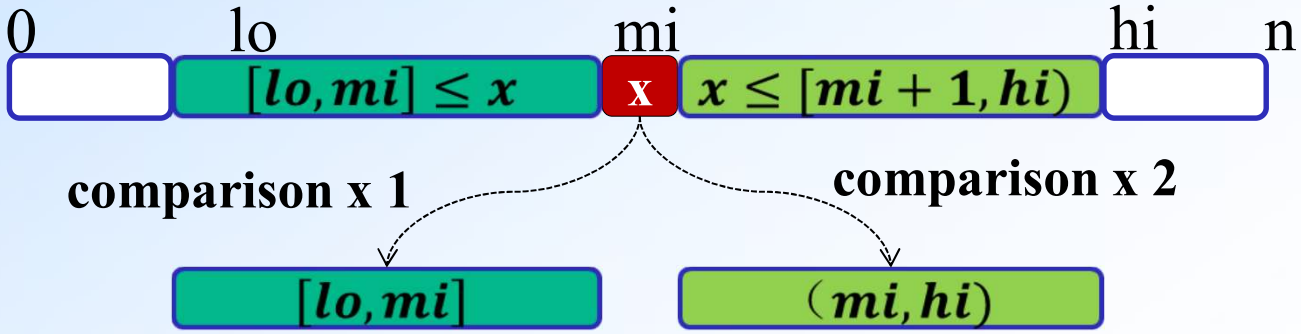
   _size = ++i;
   shrink();                                 //直接截除尾部多余元素
   return j - i;                             //向量规模变化量，即被删除元素总数
}
```



有序向量的查找

■ 二分查找

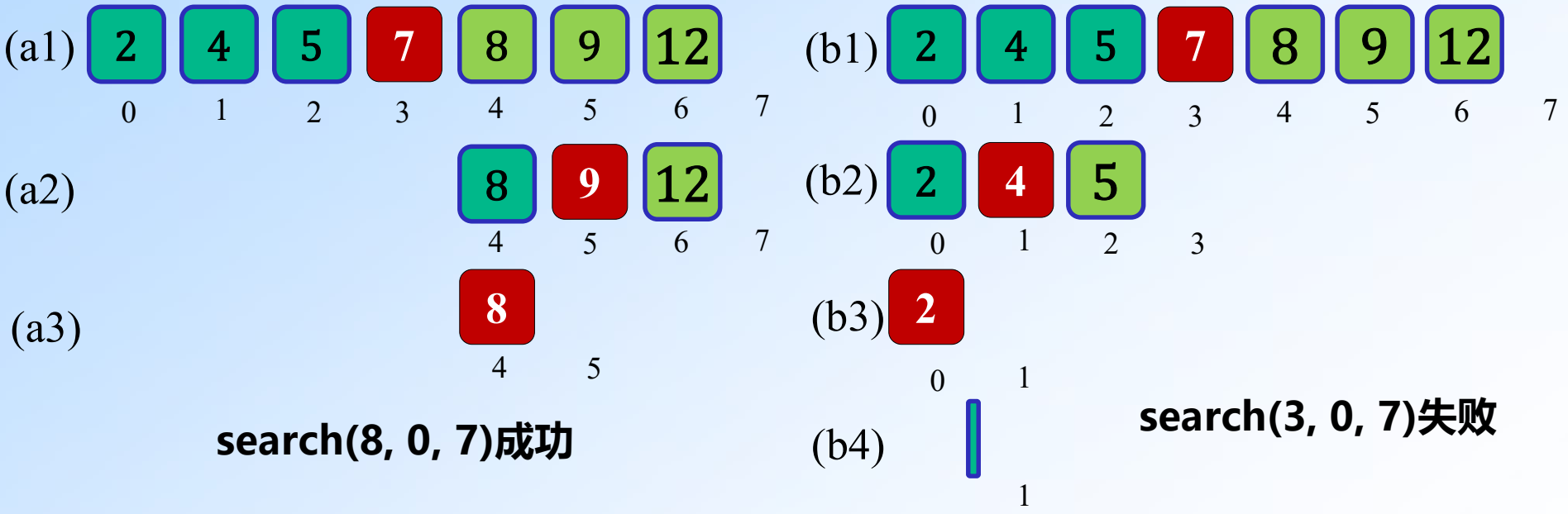
```
// 二分查找算法：在有序向量的区间[lo, hi)内查找元素e，0 <= lo <= hi <= _size
template <typename T>
static Rank binSearch ( T* A, T const& e, Rank lo, Rank hi ) {
      while ( lo < hi ) {                          //每步迭代可能做两次比较判断，三个分支
            Rank mi = ( lo + hi ) >> 1;            //以中点为轴点
            if( e < A[mi] ) hi = mi;               //深入前半段[lo, mi)继续查找
            else if ( A[mi] < e ) lo = mi + 1;     //深入后半段(mi, hi)继续查找
            else return mi;                        //在mi处命中
      }                                            //成功查找可以提前终止
      return -1;                                   //查找失败
} //多个命中元素时，不能保证返回秩最大者；查找失败时，简单地返回-1，而不能指示失败位置
```



有序顺序表的查找

- **二分查找**
  - ✓ **实例**

从[0,7) 中搜8和3



search(8, 0, 7)成功

search(3, 0, 7)失败

- ✓ 有效宽度按1/2比例以几何级数速度递减，至多经过$\log_2(hi-lo)$步迭代，算法终止
- ✓ 时间复杂度：$O(\log_2 n)$

顺序表的优缺点

优点：

- 简单、运算方便，特别是对于小线性表或长度固定的线性表，采用顺序存储结构的优越性更为突出
- 随机访问效率高

缺点：

- 顺序存储插入与删除一个元素，需要移动若干个数据元素，对大的线性表，特别元素的插入和删除频繁时，顺序存储效率低；（如果不要求顺序存储，有优化方法吗？）
- 顺序存储空间分多了浪费，分少了空间不足上溢，存储空间不便扩充

链表（Linked List）

链表是线性表的链接存储表示

- **单链表**
- **静态链表**
- **循环链表**
- **双向链表**

单链表(Singly Linked List)

定义：用一组地址任意的存储单元存放线性表中的数据元素。

例如：线性表ZHOU,ZHAO,SUN,WU,WANG,ZHANG,LI

头指针

31

表中节点位置	存储地址	数据域	指针域
6	1	ZHANG	13
5	7	WANG	1
7	13	LI	null
2	19	ZHAO	37
4	25	WU	7
1	31	ZHOU	19
3	37	SUN	25

单链表结构

每个元素由结点(Node)构成, 它包括两个域:数据域Data和指针域Link



存储结构： 链式存储结构

特点： 存储单元可以不连续。

存取方式： 顺序存取。

▪ 单链表的类型定义

```
typedef char ListData;
```
定义数据域的类型为字符型

```
typedef struct node {          //链表结点
   ListData  data;               //结点数据域
    struct node  * link;         //结点链域
} ListNode;

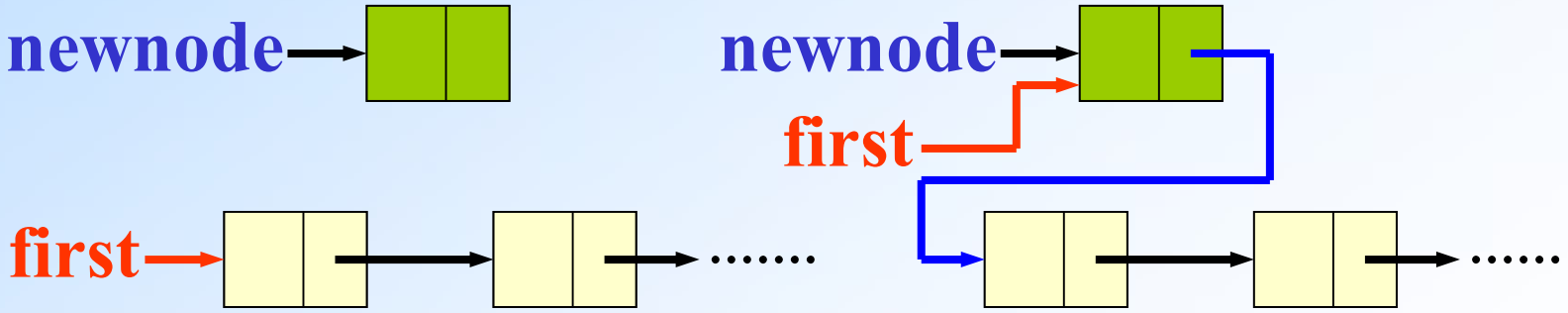
typedef ListNode  * LinkList;   //链表头指针
LinkList  first;                //链表头指针
```

- 单链表的基本运算
  - 插入（三种情况）

先连后面，再连前面

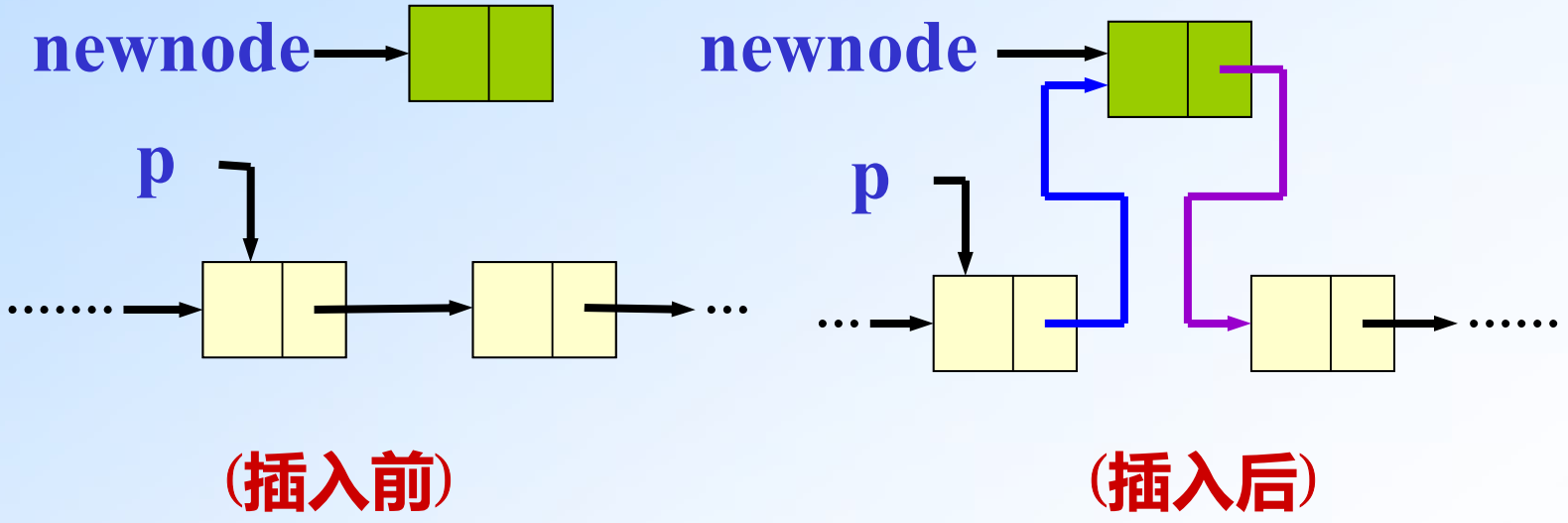
- **第一种情况：在第一个结点前插入**

```
newnode->link = first ;
first = newnode;
```



· 第二种情况：在链表中间插入

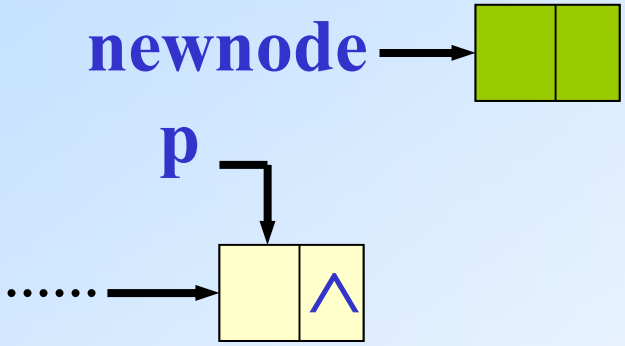
```
newnode->link = p->link;
p->link = newnode;
```



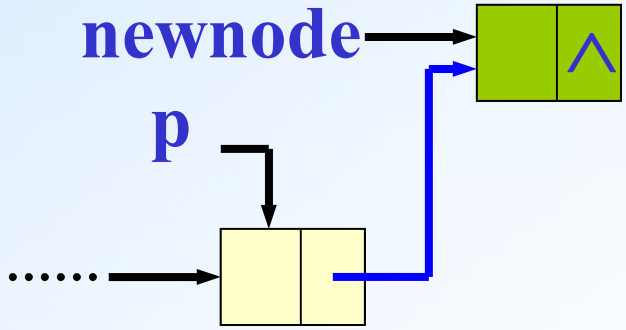
工作指针p指向将要插入位置的前驱结点

- **第三种情况：在链表末尾插入**

```
newnode->link = p->link;
p->link = newnode;
```



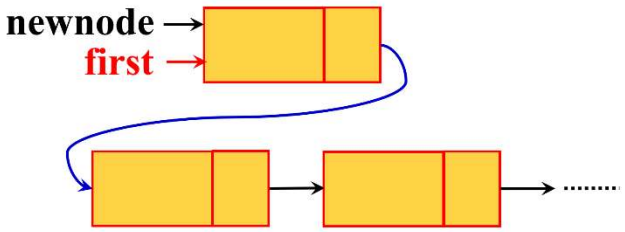
(插入前)



(插入后)

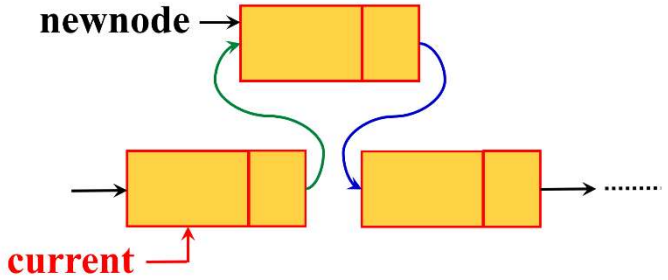
单链表的插入

■ 表头插入（无表头节点）：在列表最前端插入




```
newnode->next = first;
first = newnode;
```

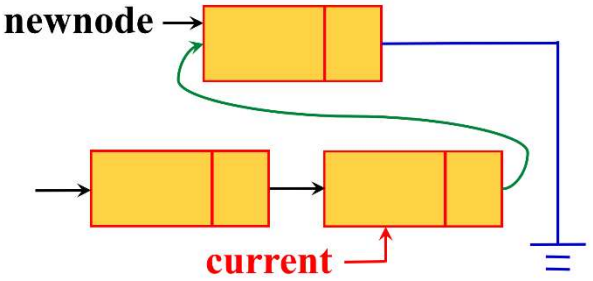
■ 中间插入（无表头节点）：在列表中间插入




```
newnode->next = current->next;
current->next = newnode;
```

后两种情况插入操作相同，但在表前端插入需单独处理

■ 末端插入（无表头节点）：在列表末尾插入




```
newnode->next = current->next;
current->next = newnode;
```

归根结底，列表需定位被插入节点（即插入节点的后一节点）的前一节点current(p)，但在无表头节点的表头插入情况下，该节点不存在

```
int ListInsert (int x, int i ) {
//在链表第 i 个结点处插入新元素 x
    ListNode * p = first;
    for ( k = 0; k < i -1; k++ )  //找第 i-1个结点
      if ( p == NULL ) break;
      else p = p->link;
    if ( p == NULL && first != NULL ) {
       printf( “无效的插入位置!\n”);
       return 0;
    }
ListNode *newnode =          //创建新结点
       (ListNode*) malloc(sizeof (ListNode));
newnode->data = x;
```

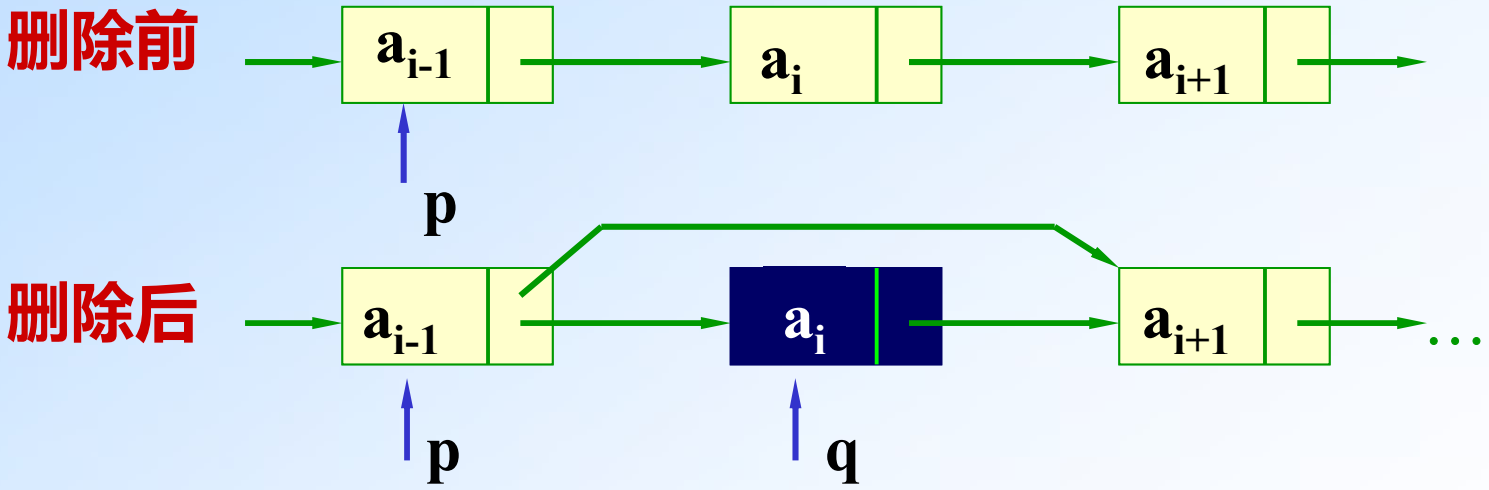
```
if ( first == NULL || i == 0 ) {  //插在表前
        newnode->link = first;
        if ( first == NULL ) last = newnode;
        first = newnode;
    }
    else {                          //插在表中或末尾
        newnode->link = p->link;
        if ( p->link == NULL ) last = newnode;  //表尾
        p ->link = newnode;
    }
    return 1;
}
```

- **删除**

在单链表中删除a_i结点

```
q = p->link;
p->link = q->link;
free(q);
```

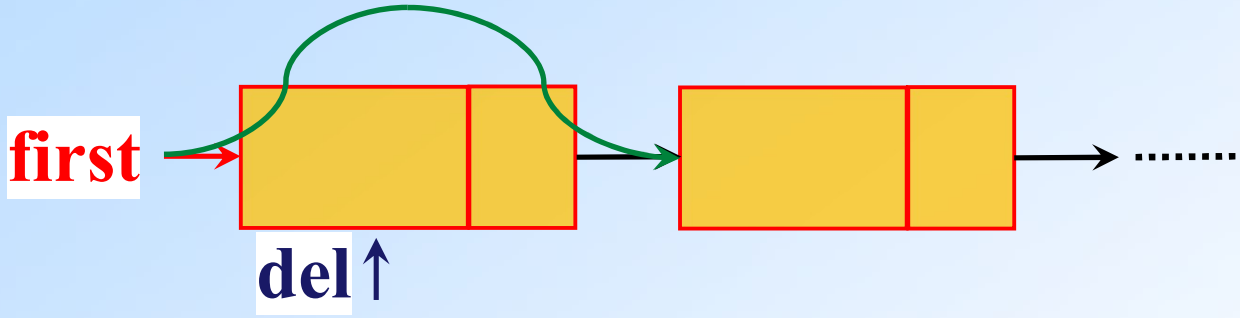
工作指针*p*指向将要删除结点的前驱结点



先摘后连，再释放

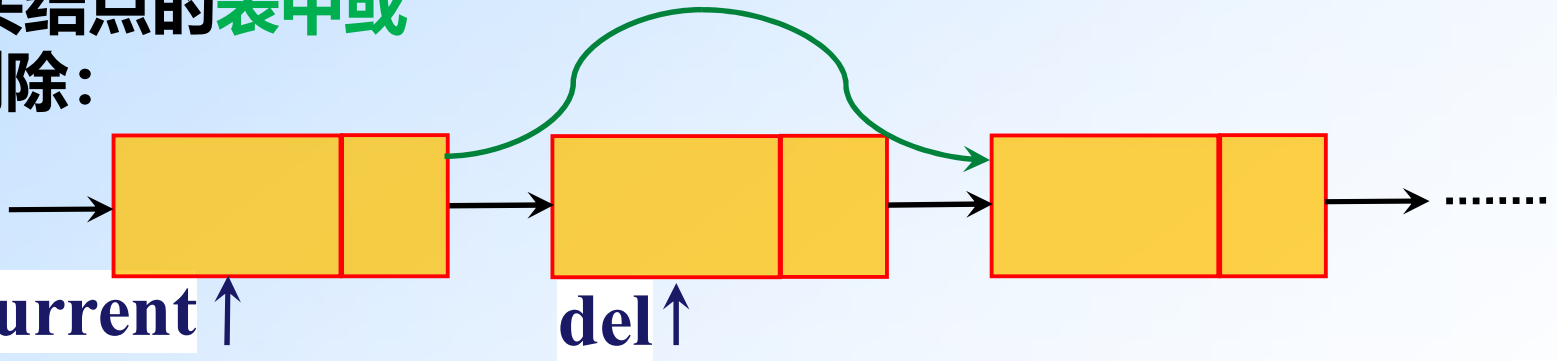
单链表的删除实现

- **无表头结点的首节点删除：**




```
del = first;   first = first->next;   delete del;
```

- **无表头结点的表中或表尾删除：**




```
del=current->next;  current->next=del->next;  delete del;
```

- ✓ 链表的插入和删除，不需移动元素，只需修改结点指针，比顺序表简易
- ✓ 表中没有表头结点，则操作稍复杂，要专门处理空表和在表头插入特殊情形
- ✓ 寻找插入或删除位置只能沿着表链检测。

```
int ListRemove ( int i ) {
//在链表中删除第 i 个结点
    ListNode *p, *q;
    if ( i == 0 )            //删除表中第 1 个结点
      { q = first;  p = first = first->link; }
    else {
      p = first;    int k = 0;    //找第 i-1个结点
      while ( p != NULL && k < i-1 )
        { p = p->link; k++; }
      if ( p == NULL || p->link == NULL ) {
        printf( "无效的删除位置!\n"); return 0;
      }
```

```
else {          //删除表中或表尾元素
    q = p->link;      //重新链接
    p->link = q->link;
}
if ( q == last ) last = p;      //可能修改last
Type x = q->data;
free(q);                //删除q
return x;          //返回第 i 个结点的值
}
```